STRIPE PROJECT

6. Machine Learning Integration

D7. Machine Learning Integration Strategy

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D7 : ML Integration Strategy / v09 --- 2026/11/05.*

D7.1. Contexte

Ce document présente l'intégration des modèles de machine learning dans le système *NoSQL* : extraction des features, déploiement des modèles, monitoring des performances.

D7.2. Principes

Cas de la détection de fraude

La production et l'exploitation des modèles de machine learning pour le scoring de fraude s'insèrent dans l'architecture globale sous une contrainte extrêmement forte :
la décision de fraude est une sortie *opérationnelle* qui doit être prise en moins de 100 millisecondes pendant que le client attend la validation de son paiement.

Cette contrainte guide toute la conception de l'intégration :

- aucune agrégation/jointure au moment de la décision ;
- les features sont continûement pré-calculés ;
- les features courants sont lus à la volée par clé unique ;
- le système *NoSQL* joue le rôle de magasin de features en ligne ;
- le modèle est servi par un service dédié ;
- le service de scoring n'est couplé qu'avec le système *NoSQL* (ni *OLTP* ni *OLAP*) ;

Autres cas d'usage

Les autres utilisations de modèles de machine learning (personnalisation, analyse prédictive) tolèrent des latences supérieures et suivent des chemins différents.

D7.3. Cas de la détection de fraude

D7.3.1. Architecture d'ensemble

Étape	Composant	Technologie	Latence
Collecte des données	Bus d'évènement	Kafka + Topics `cdc.transaction` & `user.sessions`	Continue
Calcul des features	Traitement de flux	Apache Flink	< 1 s
Magasin de features *online*	NoSQL	MongoDB + `ml_features`	Lecture < 10 ms
Magasin de features *offline*	Historique complet, requêtes analytiques	Snowflake	Lot
Service de scoring de fraude	Service dédié	API	< 50 ms
Décision de fraude	Application de paiement	S/O	< 100 ms bout en bout
Traçabilité (publication d score)	Bus d'évènement	Kafka + Topics `fraud.scores`	Continue
Monitoring	Métriques et alertes	*Prometheus*, *Grafana*	Continue

Flux nominaux (*) :

1. L'application de paiement transmet la demande de transaction :
    1. La transaction arrive dans la base *OLTP* ;
        1. Le consommateur *CDC* (*WAL+Debezium*) publie la transaction dans *Kafka* ;
    2. Parallèlement, le service de scoring :

    1. Récupère les features et les caches courants de la base *NoSQL* ;
    2. Produit un score ;
    3. Publie le score dans *Kafka* (asynchroniquement);
    4. Remonte le score synchroniquement à l'application
  3. L'application de paiement prend la décision d'autoriser, de bloquer ou de suspendre la transaction en fonction du score ;
2. Parallèlement, un consommateur *Kafka* reporte continûement les changements de score (publiés dans *Kafka*) dans la table *OLTP* `FraudScore` ;
3. Parallèlement, un job *Flink* actualise continûement les features de la base *NoSQL* (recalcule les fenêtres glissantes) ;

(*) flux détaillés de bout en bout dans ~~[D5] (§D5.5.4)~~.

D7.3.2. Extraction et gestion des features

D7.4.1 Qu'est-ce qu'un *feature*?

Un *feature* est une grandeur dérivée des données brutes, et sélectionnée pour sa valeur prédictive pour un modèle donné. Exemples :

Donnée brute (OLTP)	Feature exploitable
`created_at`	`hour_of_the_day` , `is_night` , `hours_since_last_transaction`
`ip_geolocation`	`distinct_countries_7d` , `is_country_consistent_with_card`
`amount`	`mean_ticket_deviation_90d`
`status`	`failed_ratio_7d`
`payment_method_id`	`distinct_cards_30d`

D7.4.2 Signaux pour la détection de fraude

Ecarts par rapport à l'identité

- empreinte du navigateur, cohérence entre géolocalisation et pays de la carte de paiement, usage d'un VPN, appareil inconnu.

Ecarts par rapport au comportement habituel

- montant anormal, heure inhabituelle, changement soudain d'appareil.

Comportements dynamiques inhabituels

- nombre de transactions sur fenêtre courte, cartes distinctes, pays distincts, taux d'échec récent.

D7.4.3 Calcul continu des fenêtres glissantes

Technologie : *Apache Flink*.

Les features de comportement dynamiques reposent sur des *fenêtres glissantes* (par exemple, « les 24 dernières heures ») qui doivent être recalculées en permanence. *Flink* maintient un état par
entité, intègre chaque nouvel événement et fait sortir ceux qui expirent.

C'est ce qui distingue le traitement de flux par *Flink* d'un simple consommateur *Kafka* : il possède une mémoire.

Pourquoi ne pas calculer les fenêtres glissantes à la demande (en temps réel)?
Une requête d'agrégation SQL qui doit prendre en compte des millions de lignes ne peut pas tenir un budget de l'ordre de 1/10 de seconde. Précalculer ces features déplace le coût de ces calculs hors du chemin critique.

D7.4.4 Magasin *online* avec `ml_features`

Structure détaillée de la collection dans le modèle *NoSQL* [D4].

Les points essentiels de la collection :

- *1 document par entité* (`cust:<id>` , `merch:<id>` , `card:<empreinte>`) ;
- récupération des features par *clé primaire uniquement* (pas de jointure ni d'agrégation) ;
- *hachage* des clés de partition pour une répartition uniforme ;
- garantie de cohérence *à terme* sur le chemin de décision critique avec l'option `readConcern: "majority"` .

Le schéma flexible du document est ici un avantage concret : un modèle `v4`
qui ajoute six features et en retire trois s'écrit sans migration de schéma.

Le cycle de vie d'un modèle est rapide.

D7.4.5 Magasin *offline* pour l'entrainement

Le magasin *online* ne contient que l'état courant des features. L'*entraînement* nécessite un historique complet et un profil d'accès opposé : requêtes analytiques massives, agrégations, jointures temporelles. Le magasin *offline* est le magasin des features d'entrainement.

Technologie : *Snowflake*

- Technologie d'entrepôt déjà retenue pour le système *OLAP* ([D5] §D5.11) ;
- Schéma dédié distinct du modèle *OLAP* ;
- Réutilise directement la même gouvernance (RLS, stratégie d'effacement RGPD [D5] §D5.8.3).

	Magasin online	**Magasin offline**
Contenu	État courant des features	Historique complet des features
Accès	Lecture par clé	Requêtes analytiques
Latence	< 10 ms	Minutes
Technologie	*MongoDB*	*Snowflake*
Usage	Scoring	Entraînement

Pourquoi *Feast* n'a-t-il pas été retenu?
La valeur de *Feast* est de permettre de gouverner des définitions de features partagées entre plusieurs modèles et équipes. Cette fonction n'est pas utile dans le périmètre fonctionnel du projet : on a un seul modèle de fraude et un seul job *Flink* qui calcule ses features courants (§D7.3.3).
La cohérence entre magasin online et magasin offline vient de cette source de calcul unique, écrite dans les deux magasins, pas d'une plateforme de gouvernance par-dessus.

À réévaluer si les cas d'usage ML se multiplient au-delà du scoring de fraude.

D7.4.6 Gestion du décalage entre entraînement et service

C'est la raison d'être des magasins de features. Un modèle entraîné sur des features calculées par lots, puis interrogé avec des features calculées en flux, voit en production des données qui ne ressemblent pas à celles sur lesquelles s'est entrainé. Ses prédictions se dégradent silencieusement (aucune erreur technique, juste une perte de qualité).

Mécanisme de prévention : une seule définition de feature, partagée par les deux chemins. Le magasin garantit que chaque feature signifie exactement la même chose à l'entraînement et au scoring.

D7.3.3. Détection de fraude en temps réel

D7.3.3.1 Types de fraude ciblés

Type	**Signature**
Carte volée	Incohérence géographique, appareil inconnu, montant atypique
Test de cartes	Rafale de micro-transactions, taux d'échec élevé
Prise de contrôle de compte	Comportement anormal sur un compte légitime
Fraude amicale	Aucune signature technique - traitement *a posteriori*
Fraude du marchand	Détectée au KYB et par analyse de flux
Blanchiment	Montants sous les seuils, flux circulaires

D7.3.3.2 Architecture de décision

La décision est architecturée en trois couches de filtre stratifiées :

1. **Filtre 1 : règles déterministes.**
   - Rapides, explicables, ces règles traitent les cas évidents à coût négligeable :
     - plus de *N* tentatives / minute,
     - carte sur liste noire,
     - pays interdit,

    - etc.
2. **Filtre 2 : modèle de scoring.**
    - Le modèle produit un score d'anormalité, `anomaly_score` ;
    - La valeur du score est enregistrée dans la table `FraudScore` (*OLTP*) et dans la collection `ml_features.model_scores` (*NoSQL*).
3. **Filtre 3 : décision par seuils.**
    - Traduit le score en action en fonction des seuils définis dans `merchant_config` :

Score	Action
`score` < `manual_review_threshold`	Autoriser
`score` in [`manual_review_threshold` , `auto_block_threshold`]	Authentification renforcée (*3D* (*))
`score` > `auto_block_threshold`	Bloquer

> *(*) **3-D Secure (3DS)** est un protocole d'authentification qui complète les systèmes de paiement par carte pour les achats en ligne. Il fait intervenir la banque émettrice pour vérifier l'identité du porteur avant l'autorisation, en utilisant une logique de risque et, si nécessaire, une étape d'authentification supplémentaire. Il renforce la sécurité, aide à respecter les obligations légales, comme la SCA en Europe, et transfère une partie du risque de fraude du marchand vers la banque émettrice. Source : adyen.*

Remarques

- **Le but de la détection de fraude n'est pas de bloquer *toutes* les fraudes.**
    - Une politique trop stricte produirait une grand nombre de faux positifs se traduisant par des transactions légitimes mais refusées. Sur de gros volumes de transaction, le manque à gagner des refus abusifs (vente perdue, client perdu) dépasse fréquemment le coût de la fraude elle-même. Le rôle du seuil intermédiaire dan §D7.5.2 est d'optimiser ce compromis : dans l'intervalle du seuil, on ne refuse pas la transaction, mais on demande une vérification supplémentaire.
- **Les seuils sont configurables marchand par marchand.**
    - Un vendeur de biens numériques, très exposé, tolère plus de friction qu'un commerce à faible risque. Un seuil global serait inadapté dans les deux cas.

D7.3.3.3 Cas des paiements initiés par le marchand

Lors d'un prélèvement d'abonnement les signaux comportementaux d'un utilisateur physique (clickstream, device, géolocalisation) ne sont plus disponibles.

Un modèle calibré sur des paiements initiés par le client marquerait donc systématiquement les prélèvements récurrents comme suspects.

Solution : le champ `is_merchant_initiated` de `Transaction` route la transaction vers un modèle distinct, fondé sur d'autres signaux : régularité du prélèvement, ancienneté de l'abonnement, historique d'échecs.

D7.8. Déploiement des modèles

D7.8.1. Cycle de vie

1. **Entraînement** - sur le magasin hors ligne, avec versionnement des jeux de données.
2. **Validation** - sur un échantillon temporel postérieur à celui de l'entraînement, jamais par tirage aléatoire (voir §D7.8.4).
3. **Enregistrement** - le modèle est déposé dans un registre avec sa version, ses métriques et la définition des features utilisées.
4. **Déploiement progressif** - voir §D7.8.2.
5. **Monitoring** - voir §D7.9.

D7.8.2. Stratégie de déploiement

Déploiement fantôme (*shadow*) - le nouveau modèle score en parallèle de l'ancien, sans que ses décisions soient appliquées. On compare les sorties sur du trafic réel, sans risque.

Déploiement progressif (*canary*) - activation sur une fraction croissante du trafic : 1 %, 5 %, 25 %, 100 %, avec retour arrière automatique si les métriques se dégradent.

Retour arrière immédiat - l'ancienne version reste déployée et activable. Un modèle de fraude défaillant bloque des paiements légitimes : le retour arrière doit être instantané, pas planifié.

D7.8.3. Traçabilité

Chaque score produit est associé à la version du modèle qui l'a généré :

Cas de la détection de fraude

- `FraudScore.model_version` (OLTP) ;
- `ml_features.model_scores` (NoSQL) - la version du modèle est encodée directement dans le nom de la clé (`fraud_v3` , par exemple), pas dans un champ `version` séparé. À ne pas confondre avec `ml_features.schema_version` ([D4] §D4.5.4), qui versionne la **structure** du document, pas le modèle qui a produit le score.

Sans cette traçabilité, il est impossible d'attribuer une dégradation à un déploiement, ni de justifier une décision de blocage auprès d'un marchand ou d'un régulateur.

D7.8.4. Contrainte propre aux données temporelles

La validation ne doit **jamais** se faire par tirage aléatoire.

Un modèle entraîné sur des transactions de mars et validé sur d'autres transactions de mars obtient d'excellents résultats - mais il a appris des schémas de fraude propres à cette période. La fraude évolue : les fraudeurs s'adaptent, les schémas changent.

Mécanisme correct : entraîner sur une période, valider sur la période **suivante**. C'est la seule façon de mesurer la capacité réelle du modèle à généraliser dans le temps.

D7.9. Monitoring et actualisation

D7.9.1. Métriques

- **Métriques techniques** : latence de scoring (p50, p95, p99), taux d'erreur du service, fraîcheur des features, décalage du consommateur *Flink*.
- **Métriques de qualité** : précision, rappel, taux de faux positifs et de faux négatifs, aire sous la courbe.
- **Métriques métier** : taux d'autorisation, taux de rétro-facturation, montant de fraude évitée, montant de ventes perdues par refus abusif.

Ces trois familles sont nécessaires : un modèle peut être techniquement sain, statistiquement correct, et pourtant coûteux en ventes perdues.

D7.9.2. Le problème du label différé

Difficulté structurelle de la détection de fraude : **on ne sait pas immédiatement si une transaction était frauduleuse**.

Le label vient de la rétro-facturation, qui arrive plusieurs semaines après la transaction.
Conséquences :

- la vraie performance d'un modèle n'est mesurable qu'avec ce décalage ;
- les données d'entraînement récentes sont **incomplètement labellisées** ;
- un modèle peut se dégrader pendant des semaines avant que la métrique de qualité ne le révèle.

Mécanisme de compensation : suivre en continu des **indicateurs avancés** qui ne dépendent pas du label - distribution des scores, taux de blocage, taux d'authentification renforcée. Une dérive de ces indicateurs alerte bien avant que les rétro-facturations ne confirment le problème.

D7.9.3. Dérive des données et des concepts

Dérive des données (*data drift*) : la distribution des features change : nouveau marché, nouveau type de marchand, saisonnalité. Détectée par comparaison statistique entre la distribution d'entraînement et celle observée.

Dérive du concept (*concept drift*) : la relation entre features et fraude change : les fraudeurs adaptent leurs méthodes. C'est la dérive propre à un domaine *adversarial.

Cette dimension *adversariale* distingue la fraude de la plupart des problèmes de machine learning : l'adversaire observe le système et s'y adapte. Un modèle figé perd mécaniquement en efficacité, même sans changement d'infrastructure.

D7.9.4. Politique de réentraînement

Déclencheur	Action
Périodique (mensuel)	Réentraînement systématique sur données récentes
Dérive détectée	Réentraînement anticipé
Dégradation métier	Investigation, puis retour arrière ou réentraînement
Nouveau schéma de fraude identifié	Ajout de features, réentraînement

Le réentraînement périodique n'est pas une précaution : c'est la réponse structurelle à la dérive *adversariale*.

D7.9.5. Explicabilité et conformité

Deux exigences imposent que les décisions soient explicables :

- *Vis-à-vis du marchand* : un blocage doit pouvoir être justifié. Un marchand qui perd une vente légitime attend une explication.
- *Vis-à-vis du régulateur* : le RGPD encadre la décision automatisée produisant des effets significatifs. Le droit à une explication *et à une intervention humaine* s'applique. C'est la fonction du seuil de revue manuelle : au-dessus d'un certain seuil de risque, c'est à un humain de trancher.

Mécanismes : conservation des principaux facteurs ayant contribué au score, traçabilité de la version du modèle, journalisation de chaque décision dans les structures d'audit (~~`DataAccessLog`~~ , ~~`FactAuditEvent`~~).

D7.10. Récapitulatif des technologies

Fonction	Technologie	Alternative
Calcul des features en flux	*Apache Flink*	Kafka Streams, Spark Streaming
Orchestration des définitions de features	Aucune - calcul unique via *Flink*	Feast
Magasin en ligne	*MongoDB* (`ml_features` , déjà spécifié - [D4] §D4.5.4)	Redis, DynamoDB
Magasin hors ligne	*Snowflake*, schéma dédié ([D5] §D5.11)	Parquet sur stockage objet
Entraînement	Python (*scikit-learn*, *XGBoost*)	PyTorch pour le séquentiel
Registre de modèles	*MLflow*	SageMaker Model Registry
Service de scoring	Service conteneurisé	SageMaker Endpoint, Seldon
Monitoring	*Prometheus*, *Grafana*, *Evidently*	Datadog, Arize

Note sur les sources

Le document source du projet mentionne explicitement : l'intégration de modèles de machine learning **dans le système NoSQL**, la **détection de fraude en temps réel**, la **personnalisation client**, l'**analytique prédictive**, la nécessité que le pipeline supporte l'**extraction de features** et le **déploiement de modèles**, ainsi que des mécanismes de **monitoring des performances et de mise à jour des modèles**. Il mentionne également un champ « fraud indicators » avec des scores d'anomalie.

En revanche, il ne décrit **aucun type de fraude spécifique**, aucun algorithme, aucun seuil, aucune métrique. Les types de fraude recensés en §4.1, l'architecture de décision en trois couches, la distinction magasin en ligne / hors ligne, les stratégies de déploiement, le traitement du label différé et la politique de réentraînement constituent des propositions inférées, cohérentes avec les exigences exprimées mais non prescrites par le cahier des charges.

D7.6. Autres cas d'utilisation

D7.6.1. Cas 2 : Personnalisation client

Cas d'usage

- **Segmentation comportementale** : regroupement des clients selon la fréquence des transactions, le montant moyen, l'ancienneté et la récence d'achat.
  - Exploitation : alimente la dimension analytique `DimCustomer.segment` dans l'entrepôt *OLAP* (invoqué lors de la transformation des données *OLTP*/*NoSQL* en données analytiques).
- **Recommandations** : produits susceptibles d'intéresser un client en fonction de son historique et de celui des clients similaires. Le moteur de recommandations est alimenté par la collection `user_sessions` de la base *NoSQL* (sources : clickstream, parcours, abandons) (éventuellement, le moteur exploite également les données analytiques comme l'historique des achats du client).
  - Exploitation : les applications clientes, en temps réel (publicité, etc.), ou la base analytique.
  - Flux de traitement : comme dans la détection des fraudes, les features sont pré-calculés continûement en amont, et le moteur de recommendation récupère les features courants, en temps réel, depuis la base *NoSQL*.
- **Optimisation du parcours de paiement** : prédiction du moyen de paiement le plus susceptible d'aboutir pour un client donné, afin de le proposer en premier.
  - Exploitation : les applications clientes, en temps réel, ou la base analytique.
  - Flux de traitement : même principes : les features sont pré-calculés continûement en amont, et le moteur d'optimisation récupère les features courants, en temps réel, depuis la base *NoSQL*.

Criticité

[A VOIR]

Cloisonnement

La personnalisation des services pour un client donné reste cloisonnée à un marchand, et un seul : les données comportementales du client d'un marchand ne peuvent pas alimenter les recommandations d'un autre client d'un autre marchand, même si les deux clients correspondent à la même personne physique.

Mode d'exécution

Calcul par *lots nocturne*, résultats écrits dans la base *NoSQL* et dans l'entrepôt OLA. Aucun traitement de flux nécessaire.

D7.6.2. Cas 3 : Analytique prédictive

Cas d'usage

Prédiction	Horizon	Usage
Attrition d'abonnement (*churn*)	30 jours	Action de rétention marchand
Risque de rétro-facturation	À la transaction	Provisionnement, surveillance
Prévision de revenus	Mensuel	Pilotage marchand et interne
Échec de prélèvement récurrent	À l'échéance	Relance anticipée (*dunning*)

~~Ce dernier point mérite un mot : prédire qu'un prélèvement va échouer - carte proche de l'expiration, historique d'échecs - permet de solliciter le client **avant** l'incident, plutôt que d'enclencher une procédure de relance après. C'est un cas où la prédiction évite l'événement plutôt que de le constater.~~

Mode d'exécution

Par lots, alimentant `FactSubscriptionSnapshot` et les tables d'agrégats.

Références

Spécifications

- [D0] *Stripe Project Introduction*

Livrables

- [D1] *Comprehensive Data Architecture Diagram*
- [D4] *NoSQL Data Model*
- [D5] *Data Pipeline Architecture*

Annexes

- [D2-B] *OLTP Physical Data Model*